Лабораторная работа №3

Этап 1. Развёртывание объектного хранилища RustFS

Что требовалось: развернуть на локальной машине объектное хранилище RustFS в соответствии с подходом, использованным в домашнем задании № 2.

Подготовлена конфигурация `docker-compose.yml` для запуска RustFS в контейнере. Настроили доступ к S3 API и веб-консоли, а также подключили четыре именованных тома для хранения данных.

Для доступа к хранилищу используются параметры авторизации, задаваемые через переменные окружения. Python-приложение обращается к RustFS через S3-совместимый интерфейс.

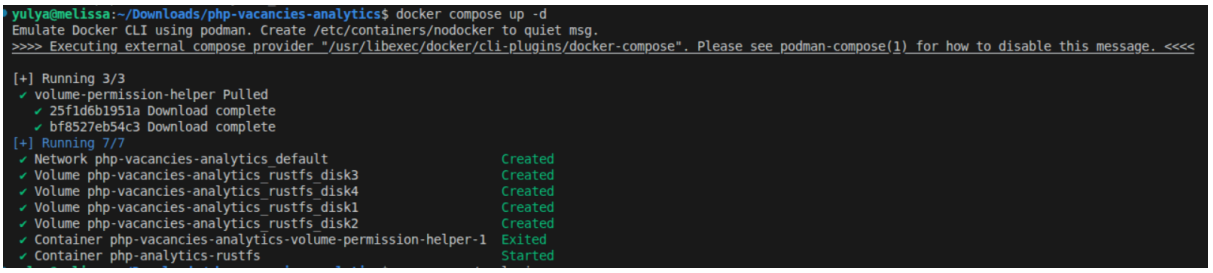
При настройке также решили проблему с правами доступа к хранилищу в Podman: настроили владельца каталогов данных через вспомогательный контейнер.

Где это находится в проекте:

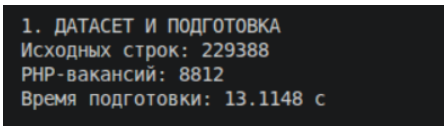
- `docker-compose.yml` — конфигурация контейнеров RustFS, портов, томов и параметров запуска.
- `.env.example` — пример переменных окружения для подключения к хранилищу.
- `src/storage.py` — создание S3-клиента и взаимодействие с RustFS.

Результат: RustFS успешно запускается локально и доступен Python-приложению. Объектное хранилище используется в последующих этапах аналитического конвейера.

Запуск докера:



Из сводки для отчета:



Этап 2. Загрузка данных

Что требовалось: загрузить в RustFS набор данных IT-вакансий, соответствующий индивидуальному варианту.

Индивидуального вариант - **5. Программист, разработчик на php.**

Исходный CSV-файл с вакансиями загружается в бакет `php-vacancies` под ключом:

`raw/vacancies.csv`

После этого выполняется подготовка данных. Программа читает исходный CSV частями, отбирает вакансии, относящиеся к PHP, проверяет необходимые поля и подготавливает данные о навыках и заработной плате.

Подготовленный набор сохраняется в RustFS под ключом:

`processed/php_vacancies.csv`

Таким образом, в хранилище находятся исходные и подготовленные данные, которые используются на последующих этапах.

Где это находится в проекте:

- `src/storage.py` — функции подключения к RustFS, загрузки и чтения объектов.
- `src/analysis.py` — подготовка данных, фильтрация PHP-вакансий и сохранение подготовленного CSV.
- `data/vacancies.csv` — локальный исходный файл, если он предусмотрен структурой проекта.

Результат: данные PHP-вакансий загружаются в RustFS и подготавливаются для дальнейшего анализа.

Этап 3. Подсчёт навыков двумя способами

Что требовалось: рассчитать частоты отдельных навыков, пар навыков и троек навыков двумя независимыми способами:

1. Собственная реализация MapReduce, читающая данные из объектного хранилища частями.
2. SQL-запросы к DuckDB, работающие с теми же данными.

3.1. Реализация MapReduce

Разработана собственную реализацию алгоритма MapReduce на Python.

На этапе **Map** программа последовательно читает подготовленный CSV непосредственно из RustFS, обрабатывая его частями. Для каждой части данных подсчитываются:

- частоты отдельных навыков;
- частоты пар навыков;
- частоты троек навыков.

На этапе **Reduce** результаты обработки отдельных частей объединяются. Для этого используются счетчики, позволяющие суммировать частоты одинаковых навыков и комбинаций.

Такой подход позволяет обрабатывать данные блоками, не загружая весь CSV в память одновременно на этапе подсчета навыков.

Результаты сохраняются в CSV-файлы.

Где это находится в проекте:

- `src/analysis.py` — функция `run_mapreduce_from_rustfs()`.
- `src/storage.py` — функция `open_object()`, обеспечивающая чтение объекта из RustFS.

3.2. Анализ с помощью DuckDB

Второй способ реализован с помощью DuckDB и SQL-запросов.

DuckDB подключается к RustFS через S3-совместимый интерфейс с использованием расширения `httpfs`. Это позволяет обращаться к подготовленному CSV в объектном хранилище, а не только к локальному файлу.

SQL-запросы рассчитывают частоты отдельных навыков, пар и троек навыков. При формировании комбинаций учитывается порядок элементов, чтобы одна и та же комбинация не учитывалась несколько раз из-за перестановки навыков.

Где это находится в проекте:

- `src/analysis.py` — функция `run_duckdb_from_rustfs()`, настройка DuckDB и запуск SQL-анализа.
- `sql/analysis.sql` — SQL-запросы для подсчёта навыков, пар и троек.

Результат этапа

Реализованы два независимых способа расчёта частот на основе одного подготовленного объекта в RustFS.

Из сводки для отчета:

```
2. ТОП-10 ОТДЕЛЬНЫХ НАВЫКОВ
--------------------------------------------------------------------------------
     skill  frequency
       PHP       7406
     MySQL       3766
       Git       2991
JavaScript       2776
      HTML       2223
   Laravel       1877
       CSS       1578
       SQL       1549
PostgreSQL       1465
1С-Битрикс       1418

3. ТОП-10 ПАР НАВЫКОВ
--------------------------------------------------------------------------------
   skill_1    skill_2  frequency
     MySQL        PHP       3596
       Git        PHP       2865
JavaScript        PHP       2675
      HTML        PHP       2154
   Laravel        PHP       1779
       Git      MySQL       1759
JavaScript      MySQL       1588
       CSS        PHP       1498
       PHP        SQL       1487
      HTML JavaScript       1459

4. ТОП-10 ТРОЕК НАВЫКОВ
--------------------------------------------------------------------------------
   skill_1    skill_2    skill_3  frequency
       Git      MySQL        PHP       1672
JavaScript      MySQL        PHP       1529
      HTML JavaScript        PHP       1415
       CSS       HTML        PHP       1231
      HTML      MySQL        PHP       1207
       Git JavaScript        PHP       1148
       CSS JavaScript        PHP       1020
   Laravel      MySQL        PHP        944
       Git       HTML        PHP        882
       CSS       HTML JavaScript        874
```

Этап 4. Верификация результатов

Что требовалось: убедиться, что результаты MapReduce и DuckDB полностью совпадают.

Реализован отдельный модуль для автоматического сравнения результатов двух методов.

После завершения расчётов программа сравнивает:

- частоты отдельных навыков;
- частоты пар навыков;
- частоты троек навыков.

Для каждой группы результатов сравниваются рассчитанные частоты соответствующих элементов. Если обнаруживаются расхождения, программа выводит информацию о различиях и завершает проверку с ошибкой.

Если все проверки успешны, выводится сообщение о совпадении результатов.

Верификация запускается автоматически при выполнении основного сценария программы после расчётов MapReduce и DuckDB.

Где это находится в проекте:

- `src/verification.py` — функции сравнения результатов.
- `src/analysis.py` — функция `main()`, в которой запускается верификация.

Результат: предусмотрена автоматизированная проверка корректности вычислений двумя независимыми способами. Она позволяет обнаруживать расхождения между результатами Python-реализации и SQL-анализа.

Для подтверждения выполнения этого этапа в отчет следует добавить фактический вывод консоли с сообщением об успешной верификации.

Этап 5. Анализ влияния навыков на заработную плату

Что требовалось: сравнить вакансии по уровню заработной платы и набору навыков, определить пары и тройки навыков, которые наиболее существенно повышают или понижают медианную зарплату.

Реализован дополнительный анализ заработной платы для подготовленного набора PHP-вакансий.

Сначала рассчитывается базовая медианная зарплата по вакансиям, для которых доступна подходящая информация о заработной плате.

Затем для комбинаций навыков рассчитываются медианные зарплаты. Полученные значения сравниваются с базовой медианой, чтобы оценить, насколько заработная плата вакансий с определёнными навыками отличается от общего уровня.

В результате можно определить:

- пары навыков, связанные с более высокой медианной зарплатой;
- тройки навыков, связанные с более высокой медианной зарплатой;
- пары навыков, связанные с более низкой медианной зарплатой;
- тройки навыков, связанные с более низкой медианной зарплатой.

При анализе учитывается минимальное количество вакансий для рассматриваемой комбинации. Это помогает не делать выводы о зарплате на основании единичных наблюдений.

Также в коде предусмотрена обработка обозначений валют RUB и RUR.

Где это находится в проекте:

- `src/analysis.py` — функция `salary_analysis()`.

Результат: реализован анализ связи между комбинациями навыков и медианной зарплатой. Он позволяет выявить сочетания навыков, для которых наблюдается более высокий или более низкий уровень оплаты.
Из сводки отчета:

```
5. ЗАРПЛАТНЫЙ АНАЛИЗ
Базовая медианная зарплата: 125,000.00 RUB
Вакансий с учтённой зарплатой: 4737
Минимальное число вакансий для комбинации: 10

наибольшая медиана относительно базы: комбинации из 2 навыков
--------------------------------------------------------------------------------
 skill_1  skill_2  vacancy_count  median_salary  delta_vs_base
Highload RabbitMQ             16       350000.0       225000.0
Highload  Symfony             15       350000.0       225000.0
     API   Golang             10       350000.0       225000.0
  Docker Highload             16       350000.0       225000.0
      Go     PHP7             10       350000.0       225000.0
Highload    MySQL             18       350000.0       225000.0
Highload      PHP             22       350000.0       225000.0
      Go   Golang             12       350000.0       225000.0
   SOLID     Yii2             20       350000.0       225000.0
Highload    SOLID             15       350000.0       225000.0

наименьшая медиана относительно базы: комбинации из 2 навыков
--------------------------------------------------------------------------------
          skill_1 skill_2  vacancy_count  median_salary  delta_vs_base
1С: Предприятие 8      C#             10        50000.0       -75000.0
1С: Предприятие 8  Python             13        50000.0       -75000.0
1С: Предприятие 8     C++             13        50000.0       -75000.0
          1С: ERP  Python             10        50000.0       -75000.0
1С: Предприятие 8    HTML             12        50000.0       -75000.0
1С: Предприятие 8    Java             11        50000.0       -75000.0
              C++  Python             42        50000.0       -75000.0
              C++    HTML             27        50000.0       -75000.0
1С: Предприятие 8     SQL             17        50000.0       -75000.0
             CSS3   Figma             19        50000.0       -75000.0

наибольшая медиана относительно базы: комбинации из 3 навыков
--------------------------------------------------------------------------------
 skill_1  skill_2            skill_3  vacancy_count  median_salary  delta_vs_base
      Go   Golang                PHP             10       350000.0       225000.0
RabbitMQ    SOLID               Yii2             15       350000.0       225000.0
RabbitMQ    SOLID                ООП             21       350000.0       225000.0
    PHP7 RabbitMQ                ООП             15       350000.0       225000.0
Highload RabbitMQ               Yii2             14       350000.0       225000.0
   SOLID  Symfony                ООП             18       350000.0       225000.0
Highload RabbitMQ                ООП             14       350000.0       225000.0
     API   Golang                PHP             10       350000.0       225000.0
     PHP  Symfony Управление командой             12       350000.0       225000.0
Highload    MySQL             Symfony             14       350000.0       225000.0

наименьшая медиана относительно базы: комбинации из 3 навыков
--------------------------------------------------------------------------------
          skill_1 skill_2 skill_3  vacancy_count  median_salary  delta_vs_base
1С: Предприятие 8      C#     SQL             10        50000.0       -75000.0
1С: Предприятие 8      C#  Python             10        50000.0       -75000.0
1С: Предприятие 8      C#     PHP             10        50000.0       -75000.0
1С: Предприятие 8      C#    Java             10        50000.0       -75000.0
1С: Предприятие 8      C#    HTML             10        50000.0       -75000.0
1С: Предприятие 8      C#     C++             10        50000.0       -75000.0
          1С: ERP  Python     SQL             10        50000.0       -75000.0
          1С: ERP     PHP  Python             10        50000.0       -75000.0
          1С: ERP    Java  Python             10        50000.0       -75000.0
             CSS3  jQuery Webpack             11        50000.0       -75000.0
```

Этап 6. Региональный анализ

Что требовалось: выбрать три региона и сравнить их по ключевым показателям, включая зарплаты и востребованность навыков.

Реализован региональный анализ PHP-вакансий.

Программа определяет три региона с наибольшим количеством вакансий в подготовленном наборе данных.

Для каждого региона рассчитываются ключевые показатели:

- количество вакансий;
- медианная заработная плата;
- наиболее распространённые навыки.

Полученные результаты позволяют сравнить региональные рынки вакансий и определить, какие навыки чаще встречаются в каждом из выбранных регионов.

Где это находится в проекте:

- `src/analysis.py` — функция `regional_analysis()`.

Результат: реализовано сравнение трёх регионов по количеству вакансий, медианной зарплате и востребованным навыкам.

Сравнение регионов из сводки:

```
6. СРАВНЕНИЕ РЕГИОНОВ
          area  vacancy_count  median_salary  vacancies_with_salary
        Москва           3154       170000.0                   1551
Санкт-Петербург           1171       150000.0                    711
   Екатеринбург            334       125000.0                    238

Самые востребованные навыки: Москва
     skill  frequency
       PHP       2543
     MySQL       1264
       Git       1000
JavaScript        772
   Laravel        688
PostgreSQL        628
      HTML        582
    Docker        529
       SQL        515
   Symfony        470

Самые востребованные навыки: Санкт-Петербург
     skill  frequency
       PHP        980
     MySQL        501
       Git        390
JavaScript        341
   Laravel        286
      HTML        247
PostgreSQL        215
   Symfony        199
       SQL        199
    Docker        199

Самые востребованные навыки: Екатеринбург
     skill  frequency
       PHP        281
     MySQL        161
JavaScript        115
      HTML         97
       Git         95
       CSS         70
   Laravel         56
PostgreSQL         55
       ООП         51
    Docker         50
```

Дополнительные сведения и артефакты

Полный вывод работы программы с подсчетом пар и троек, сравнением способов, зарплатным анализом и сравнением регионов:

```
yulya@melissa:~/Downloads/php-vacancies-analytics$ uv run src/analysis.py
1. Проверка RustFS и загрузка исходного датасета
Загружено в RustFS: s3://php-vacancies/raw/vacancies.csv (847,619,384 байт)

2. Подготовка данных: чтение исходного объекта из RustFS чанками
Загружено в RustFS: s3://php-vacancies/processed/php_vacancies.csv (31,438,501 байт)
Исходных строк: 229,388; PHP-вакансий: 8,812; подготовка: 13.11 с

3. MapReduce: прямое потоковое чтение обработанного объекта из RustFS
Загружено в RustFS: s3://php-vacancies/results/mapreduce_skills.csv (42,452 байт)
Загружено в RustFS: s3://php-vacancies/results/mapreduce_pairs.csv (1,204,529 байт)
Загружено в RustFS: s3://php-vacancies/results/mapreduce_triplets.csv (13,594,708 байт)
{
  "method": "MapReduce (S3 streaming + chunks)",
  "seconds": 1.4566,
  "python_peak_memory_mb": 67.347,
  "rss_delta_mb": 58.859,
  "unique_skills": 1933,
  "unique_pairs": 36261,
  "unique_triplets": 308638
}

4. DuckDB: SQL по тому же объекту в RustFS
Загружено в RustFS: s3://php-vacancies/results/duckdb_skills.csv (42,452 байт)
Загружено в RustFS: s3://php-vacancies/results/duckdb_pairs.csv (1,204,529 байт)
Загружено в RustFS: s3://php-vacancies/results/duckdb_triplets.csv (13,594,708 байт)
{
  "method": "DuckDB SQL (S3 object scan)",
  "seconds": 2.9583,
  "rss_delta_mb": 0.0
}

5. Проверка результатов
OK: Отдельные навыки; совпало комбинаций: 1933
OK: Пары навыков; совпало комбинаций: 36261
OK: Тройки навыков; совпало комбинаций: 308638
Верификация пройдена: результаты MapReduce и DuckDB совпадают.

6. Зарплатный анализ
Загружено в RustFS: s3://php-vacancies/results/salary_bottom_2.csv (121,492 байт)
Загружено в RustFS: s3://php-vacancies/results/salary_top_2.csv (121,492 байт)
Загружено в RustFS: s3://php-vacancies/results/salary_bottom_3.csv (720,133 байт)
Загружено в RustFS: s3://php-vacancies/results/salary_top_3.csv (720,133 байт)
Базовая медианная зарплата: 125000.00 RUB; время: 1.19 с

7. Сравнение регионов
Загружено в RustFS: s3://php-vacancies/results/regional_summary.csv (177 байт)
Загружено в RustFS: s3://php-vacancies/results/regional_skills.csv (81,147 байт)
Региональная сводка:
           area  vacancy_count  median_salary  vacancies_with_salary
         Москва           3154       170000.0                   1551
Санкт-Петербург           1171       150000.0                    711
   Екатеринбург            334       125000.0                    238
Загружено в RustFS: s3://php-vacancies/results/profile.json (1,326 байт)

Готово. Результаты находятся в results/ и в RustFS в префиксе results/.
```

Вывод отчета по производительности и сохраненным файлам:

```
7. ПРОИЗВОДИТЕЛЬНОСТЬ
MapReduce: 1.4566 с; пиковая память Python: 67.347 MB; изменение RSS: 58.859 MB
DuckDB: 2.9583 с; изменение RSS: 0.0 MB
По общему измеренному времени MapReduce выполнился примерно в 2.03 раза быстрее.
Примечание: время DuckDB включает настройку соединения, httpfs, выполнение SQL и запись результатов.
RSS delta — разница использования памяти до и после функции, а не пиковое потребление памяти DuckDB.

8. ПРОВЕРКА СОХРАНЁННЫХ ФАЙЛОВ
OK: php_vacancies.csv (31 438 501 байт)
OK: mapreduce_skills.csv (42 452 байт)
OK: mapreduce_pairs.csv (1 204 529 байт)
OK: mapreduce_triplets.csv (13 594 708 байт)
OK: duckdb_skills.csv (42 452 байт)
OK: duckdb_pairs.csv (1 204 529 байт)
OK: duckdb_triplets.csv (13 594 708 байт)
OK: salary_top_2.csv (121 492 байт)
OK: salary_bottom_2.csv (121 492 байт)
OK: salary_top_3.csv (720 133 байт)
OK: salary_bottom_3.csv (720 133 байт)
OK: regional_summary.csv (177 байт)
OK: regional_skills.csv (81 147 байт)
OK: profile.json (1 326 байт)
```

Общий вывод

В ходе выполнения домашнего задания был подготовлен аналитический конвейер для исследования вакансий PHP-разработчиков с использованием RustFS, Python, MapReduce и DuckDB.

Реализованы загрузка и подготовка данных, подсчёт частот навыков и их комбинаций двумя независимыми способами, автоматическая верификация результатов, анализ зарплат и сравнение регионов. Дополнительно предусмотрено профилирование времени выполнения и использования памяти.